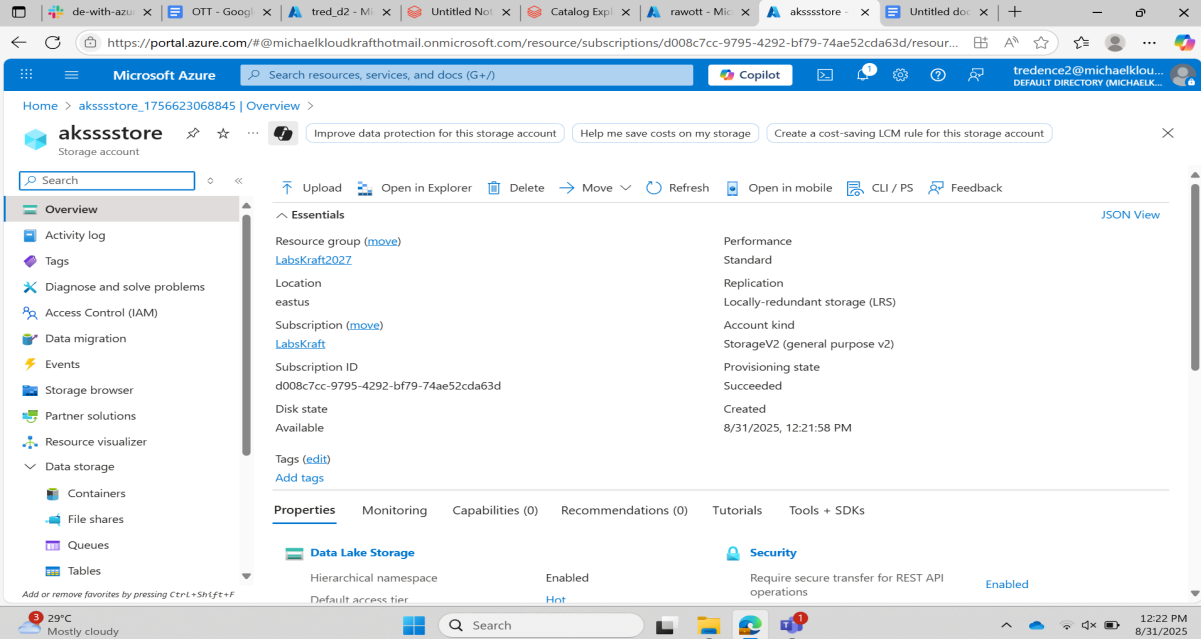


OTT Streaming Platform Analytics – Data Engineering Documentation

GROUP 2 - Akshita Bhamidimarri

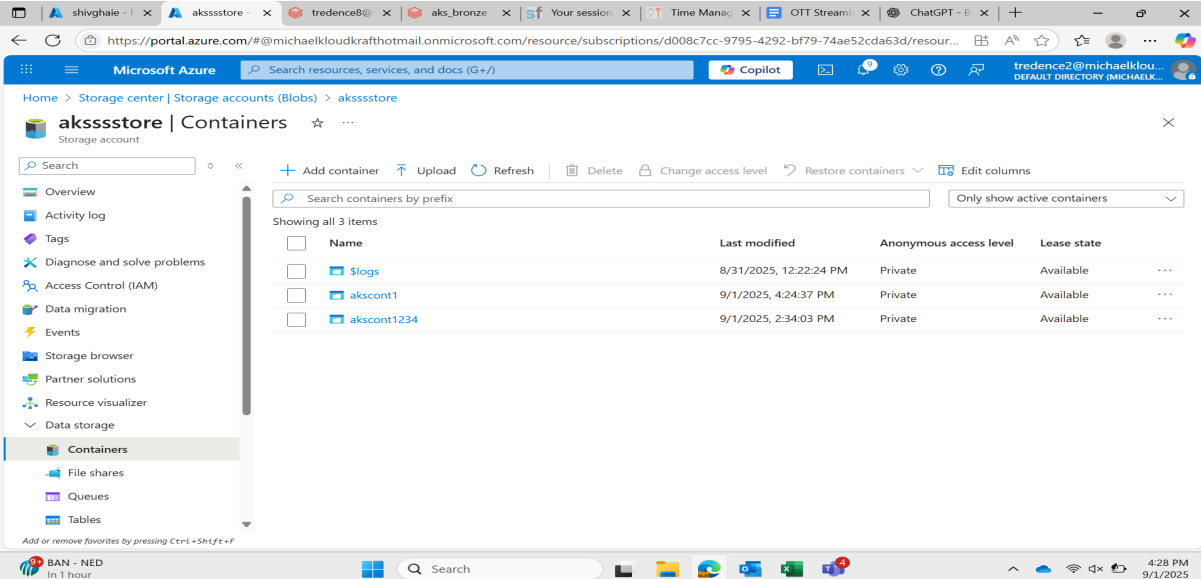
1) Create a storage account.



The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and a user profile. The main content area displays the 'Overview' page for the 'aksssstore' storage account. The left sidebar contains a navigation menu with options like Overview, Activity log, Tags, Diagnose and solve problems, Access Control (IAM), Data migration, Events, Storage browser, Partner solutions, Resource visualizer, Data storage, Containers, File shares, Queues, and Tables. The main content area shows the 'Essentials' section with details about the storage account, including its resource group, location, subscription, and disk state. Below this, there are tabs for Properties, Monitoring, Capabilities (0), Recommendations (0), Tutorials, and Tools + SDKs. The 'Data Lake Storage' section is also visible, showing the hierarchical namespace and default access tier. The 'Security' section indicates that secure transfer for REST API operations is enabled.

Microsoft Azure portal interface showing the Overview page for the 'aksssstore' storage account. The page displays various details including Resource group (move), Location (eastus), Subscription (move), Subscription ID (d008c7cc-9795-4292-bf79-74ae52cda63d), Disk state (Available), and Tags (edit). The page also shows the Data Lake Storage section with Hierarchical namespace and Default access tier, and the Security section with Require secure transfer for REST API operations.

2) Create a container inside the storage account.



The screenshot shows the Microsoft Azure portal interface, specifically the 'Containers' page for the 'aksssstore' storage account. The left sidebar contains a navigation menu with options like Overview, Activity log, Tags, Diagnose and solve problems, Access Control (IAM), Data migration, Events, Storage browser, Partner solutions, Resource visualizer, Data storage, Containers, File shares, Queues, and Tables. The main content area shows the 'Containers' section with a table listing the containers. The table has columns for Name, Last modified, Anonymous access level, and Lease state. The containers listed are '\$logs', 'akscont1', and 'akscont1234'. The page also includes a search bar for containers and a dropdown menu to show only active containers.

Microsoft Azure portal interface showing the Containers page for the 'aksssstore' storage account. The page displays a list of containers with columns: Name, Last modified, Anonymous access level, and Lease state. The containers listed are:

Name	Last modified	Anonymous access level	Lease state
\$logs	8/31/2025, 12:22:24 PM	Private	Available
akscont1	9/1/2025, 4:24:37 PM	Private	Available
akscont1234	9/1/2025, 2:34:03 PM	Private	Available

3) **Create separate folders for the ads data, content data, events data and users data inside a raw data folder in the container and add all the csv, json, parquet files in separate folders.**

The screenshot shows the Microsoft Azure portal interface. The breadcrumb navigation is 'Home > aksstore | Containers > akscont1234'. The left sidebar shows 'Overview', 'Diagnose and solve problems', 'Access Control (IAM)', and 'Settings'. The main content area shows the 'raw_data' folder under 'akscont1234'. The authentication method is 'Access key'. A search bar for blobs is present. A table lists the contents of the 'raw_data' folder:

	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	[.]					...
☐	ads_data	9/1/2025, 2:36:00 PM				...
☐	content_data	9/1/2025, 2:36:08 PM				...
☐	events_data	9/1/2025, 2:36:16 PM				...
☐	user_data	9/1/2025, 2:36:23 PM				...

4) **Create an azure connector for databricks.**

The screenshot shows the Microsoft Azure portal interface for the 'aksconnect' resource. The breadcrumb navigation is 'Home > AccessConnectorCreate-20250831124612 | Overview > LabsKraft2027 > aksconnect'. The left sidebar shows 'Overview', 'Activity log', 'Access control (IAM)', 'Tags', 'Resource visualizer', 'Settings', 'Automation', and 'Help'. The main content area shows the 'Overview' tab for 'aksconnect'. The 'Essentials' section displays the following details:

- Resource group (move): [LabsKraft2027](#)
- Location: East US
- Subscription (move): [LabsKraft](#)
- Subscription ID: d008c7cc-9795-4292-bf79-74ae52cda63d
- Tags (edit): [Add tags](#)
- State: Succeeded
- Resource ID: /subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

5) Give the azure connector Storage Blob Data Contributor access.

The screenshot shows the Microsoft Azure portal interface. The left sidebar contains navigation options like Overview, Activity log, Tags, and Access Control (IAM). The main content area displays the 'Access Control (IAM)' page for the 'aksstore' storage account. It shows a list of role assignments with columns for role, user, and resource group. The 'Storage Blob Data Contributor' role is assigned to the 'aksconnect' managed identity.

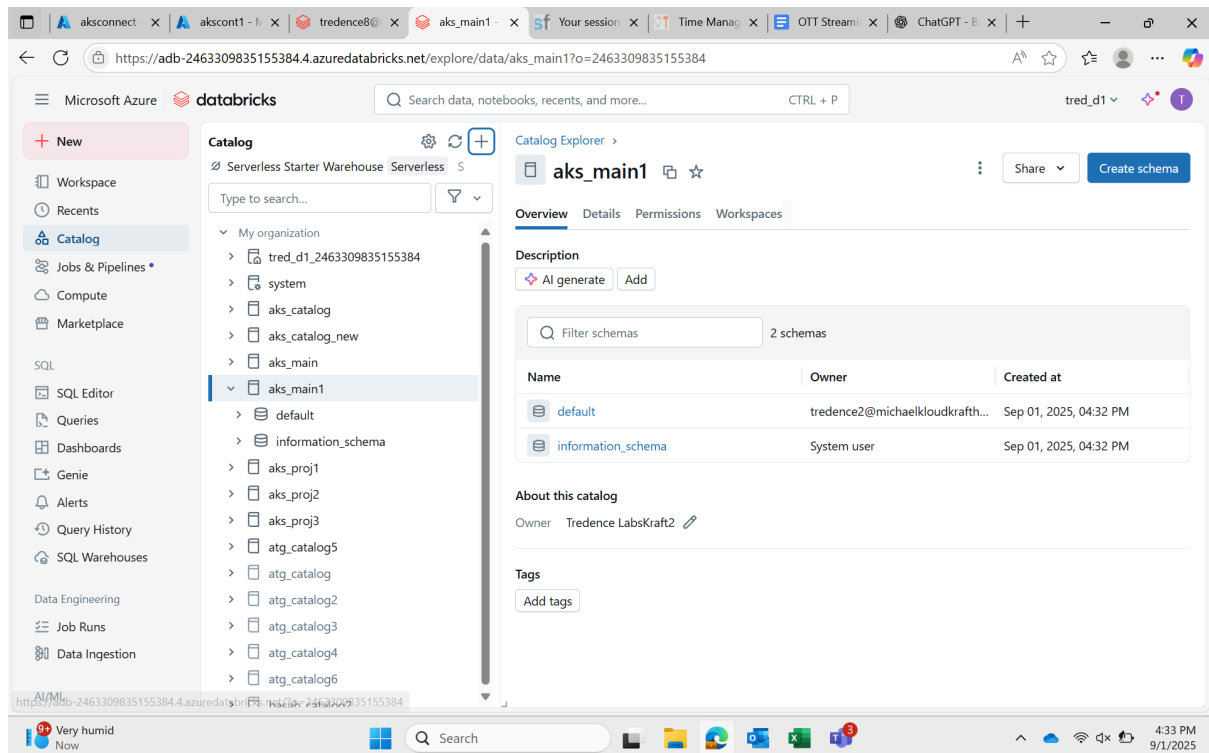
Role	User	Resource group
Owner	Tredence LabsKraft8	Resource group (Inher...)
Owner	Tredence LabsKraft9	Resource group (Inher...)
Contributor	vikram	Subscription (Inherited)
Monitoring Contributor	SherlockShinde	Subscription (Inherited)
Storage Blob Data Contributor	aksconnect	This resource
User Access Administrator	Michael A	Root (Inherited)

6) Create a new external location for the new unity catalog that we'll be using.

The screenshot shows the Databricks Catalog Explorer interface. The left sidebar contains navigation options like Workspace, Recents, Catalog, and Compute. The main content area displays the 'aks_main1' external location configuration. It includes a description, a table with credential, URL, and limit to read-only use, and a fallback mode toggle.

Credential	URL	Limit to read-only use
aks_main1_azuremanagedidentity_1756724521497	abfss://akscont1@aksstore.dfs.core.windows.net/	Disabled

7) Create the new unity catalog using the newly created external location.



8) Bronze layer code

1. **Catalog and Schemas Setup:**

Uses the catalog `aks_main`.

Creates three schemas (databases) if they don't exist: bronze, silver, gold.

Sets the current database to bronze.

2. **Base Paths:**

Defines base paths for raw data, schema locations, and checkpoints in Azure Data Lake Storage (ADLS).

3. **Schema Definitions:**

Defines explicit schemas for users, content, ads, and events datasets using Spark's StructType.

4. **Bronze Layer Data Ingestion:**

Uses Spark Autoloader (`cloudFiles`) to ingest raw data files from ADLS in **streaming mode with trigger once**, meaning the stream processes all available files once and then stops.

Handles multiple formats for each dataset: CSV, JSON, and Parquet.

Adds a metadata column `ingest_file` to track the file source.

5. **Streams for Each Dataset:**

Reads user data from CSV, JSON, and Parquet files into separate bronze tables.

Reads content data from CSV, JSON, and Parquet files into separate bronze tables.

Reads ads data from CSV, JSON, and Parquet files into separate bronze tables.

Reads event data from CSV files into a bronze table.

6. Data Storage:

Writes the ingested streaming data to Delta Lake tables under the bronze schema. Uses checkpoints for each stream to keep track of progress and support fault tolerance.

7. Trigger Once Mode:

Each stream runs once, processes all available data, and stops, suitable for batch-like ingestion but using streaming APIs for scalable file discovery.

```
import json
from pyspark.sql.types import StructType
from pyspark.sql.functions import col

# -----
# Step 0: Setup
# -----
catalog = "aks_main"

spark.sql(f"USE CATALOG {catalog}")
spark.sql("CREATE SCHEMA IF NOT EXISTS bronze")
spark.sql("CREATE SCHEMA IF NOT EXISTS silver")
spark.sql("CREATE SCHEMA IF NOT EXISTS gold")
spark.sql("USE bronze")

# Base paths in ADLS
base_path =
"abfss://akscont1234@aksssstore.dfs.core.windows.net/raw_data"
schema_base_path =
"abfss://akscont1234@aksssstore.dfs.core.windows.net/schemas"
chkpt_base_path =
"abfss://akscont1234@aksssstore.dfs.core.windows.net/chkpts"

# -----
# Step 1: Define Schemas
# -----

# Users
user_schema = StructType() \
    .add("user_id", "string") \
    .add("plan_type", "string") \
    .add("billing_cycle", "string") \
    .add("payment_status", "string") \
    .add("churn_flag", "string")
```

```

# Content
content_schema = StructType() \
    .add("content_id", "string") \
    .add("title", "string") \
    .add("genre", "string") \
    .add("duration_min", "string") \
    .add("region_availability", "string")

# Ads
ads_schema = StructType() \
    .add("ad_id", "string") \
    .add("campaign", "string") \
    .add("content_id", "string") \
    .add("impressions", "string") \
    .add("clicks", "string") \
    .add("revenue", "string")

```

```

# -----
# Step 2: Bronze Ingestion (9 Streams with trigger once)
# -----

from pyspark.sql.functions import col

# Users CSV - streaming with trigger once
bronze_users_csv = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
        f"{schema_base_path}/users_csv")
    .schema(user_schema)
    .load(f"{base_path}/user_data/csv")
    .withColumn("ingest_file", col("_metadata.file_path"))
)

# Write stream with trigger once
query = (bronze_users_csv.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/users_csv")

```

```

        .outputMode("append")
        .trigger(once=True) # process files once and stop
        .toTable(f"{catalog}.bronze.bronze_users_csv")
    )

# Wait for this stream to finish
query.awaitTermination()

```

```

from pyspark.sql.functions import col

# -----
# Users JSON - streaming with trigger once
# -----
bronze_users_json = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json") # autoloader JSON
    .option("cloudFiles.schemaLocation",
f"{schema_base_path}/users_json") # checkpoint for schema evolution
    .schema(user_schema)
    .load(f"{base_path}/user_data/json") # path to JSON files
)

# Write stream with trigger once
query_json = (bronze_users_json.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/users_json") #
checkpoint for stream progress
    .outputMode("append")
    .trigger(once=True) # process all available files once and stop
    .toTable(f"{catalog}.bronze.bronze_users_json")
)

```

```

from pyspark.sql.functions import col

# -----
# Users Parquet - streaming with trigger once
# -----
bronze_users_parquet = (spark.readStream
    .format("cloudFiles")

```

```

        .option("cloudFiles.format", "parquet")      # autoloader parquet
        .option("cloudFiles.schemaLocation",
f"{schema_base_path}/users_parquet") # schema checkpoint
        .load(f"{base_path}/user_data/parquet")      # path to parquet files
        .withColumn("ingest_file", col("_metadata.file_path"))
    )

# Write stream with trigger once
query_parquet = (bronze_users_parquet.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/users_parquet")
# stream checkpoint
    .outputMode("append")
    .trigger(once=True)      # process all available parquet files once
    .toTable(f"{catalog}.bronze.bronze_users_parquet")
)

```

```

from pyspark.sql.functions import col

# -----
# Content CSV - streaming with trigger once
# -----

bronze_content_csv = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
f"{schema_base_path}/content_csv")
    .schema(content_schema)
    .load(f"{base_path}/content_data/csv")
    .withColumn("ingest_file", col("_metadata.file_path"))
)

query_content_csv = (bronze_content_csv.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/content_csv")
    .outputMode("append")
    .trigger(once=True)
    .toTable(f"{catalog}.bronze.bronze_content_csv")
)

```



```

from pyspark.sql.functions import col

# -----
# Content JSON - streaming with trigger once
# -----

bronze_content_json = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation",
f"{schema_base_path}/content_json")
    .schema(content_schema)
    .load(f"{base_path}/content_data/json")
)

query_content_json = (bronze_content_json.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/content_json")
    .outputMode("append")
    .trigger(once=True)
    .toTable(f"{catalog}.bronze.bronze_content_json")
)

```

```

from pyspark.sql.functions import col

# -----
# Content Parquet - streaming with trigger once
# -----

bronze_content_parquet = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "parquet")
    .option("cloudFiles.schemaLocation",
f"{schema_base_path}/content_parquet")
    .load(f"{base_path}/content_data/parquet")
    .withColumn("ingest_file", col("_metadata.file_path"))
)

query_content_parquet = (bronze_content_parquet.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/content_parquet")
)

```

```

        .outputMode("append")
        .trigger(once=True)
        .toTable(f"{catalog}.bronze.bronze_content_parquet")
    )

```

```

from pyspark.sql.functions import col

# -----
# Ads CSV - streaming with trigger once
# -----

bronze_ads_csv = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation", f"{schema_base_path}/ads_csv")
    .schema(ads_schema)
    .load(f"{base_path}/ads_data/csv")
    .withColumn("ingest_file", col("_metadata.file_path"))
)

query_ads_csv = (bronze_ads_csv.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/ads_csv")
    .outputMode("append")
    .trigger(once=True)
    .toTable(f"{catalog}.bronze.bronze_ads_csv")
)

```

```

from pyspark.sql.functions import col

# -----
# Ads JSON - streaming with trigger once
# -----

bronze_ads_json = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation",
f"{schema_base_path}/ads_json")
    .schema(ads_schema)
    .load(f"{base_path}/ads_data/json")
)

```

```

query_ads_json = (bronze_ads_json.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/ads_json")
    .outputMode("append")
    .trigger(once=True)
    .toTable(f"{catalog}.bronze.bronze_ads_json")
)

```

```

from pyspark.sql.functions import col

# -----
# Ads Parquet - streaming with trigger once
# -----

bronze_ads_parquet = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "parquet")
    .option("cloudFiles.schemaLocation",
f"{schema_base_path}/ads_parquet")
    .load(f"{base_path}/ads_data/parquet")
    .withColumn("ingest_file", col("_metadata.file_path"))
)

query_ads_parquet = (bronze_ads_parquet.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/ads_parquet")
    .outputMode("append")
    .trigger(once=True)
    .toTable(f"{catalog}.bronze.bronze_ads_parquet")
)

```

```

from pyspark.sql.functions import col
from pyspark.sql.types import *

# -----
# Events CSV - streaming with trigger once
# -----

# Define schema for Events (all as StringType in Bronze)
events_schema = StructType([
    StructField("user_id", StringType(), True),

```

```

        StructField("content_id", StringType(), True),
        StructField("start_time", StringType(), True),
        StructField("end_time", StringType(), True),
        StructField("device", StringType(), True),
        StructField("geo", StringType(), True),
        StructField("bitrate", StringType(), True),
        StructField("errors", StringType(), True)
    ])

bronze_events_csv = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
        f"{schema_base_path}/events_csv")
    .schema(events_schema)
    .load(f"{base_path}/events_data")
    .withColumn("ingest_file", col("_metadata.file_path"))
    )

query_events_csv = (bronze_events_csv.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/events_csv")
    .outputMode("append")
    .trigger(once=True)
    .toTable(f"{catalog}.bronze.bronze_events_csv")
    )

```

9) Silver layer code

1. User Data (silver_users)

- Loaded raw user data from bronze table.
- Trimmed whitespaces in all string columns.
- Filtered out rows with null or invalid user IDs (starting with "invalid").
- Standardized plan_type, billing_cycle, and payment_status columns; replaced invalid/missing values with "unknown".
- Converted payment_status to lowercase.
- Created churn_flag as integer: 0 if payment status is "active", otherwise 1.
- Cast columns to correct data types before saving to the silver table.

2. Content Data (silver_content)

Loaded raw content data.
Trimmed whitespaces in string columns.
Filtered out rows with null or invalid content IDs.
Replaced invalid/null/blank title, genre, and region_availability with "unknown".
Validated and cleaned duration_min column: replaced invalid or negative durations with 0.
Filtered out duration outliers (values > 300 minutes).
Dropped unnecessary columns before saving cleaned data.

3. Ads Data (silver_ads)

Trimmed all string columns.
Removed rows with null, blank, or invalid ad_id or campaign.
Cleaned content_id, replacing invalid/null/blank with "unknown".
Validated and cast revenue, impressions, and clicks to numeric types; replaced invalid values with 0.
Ensured clicks is never greater than impressions.
Calculated average and standard deviation for impressions and clicks.
Replaced outliers (values > 3 standard deviations from mean) with average values.
Dropped unnecessary columns and saved cleaned data.

4. Events Data (silver_events)

Loaded streaming events data with predefined schema.
Trimmed whitespaces from all columns.
Filtered out rows with null or invalid IDs (event_id, user_id, content_id).
Parsed start_time and end_time into timestamp format; dropped rows missing both.
For sessions with valid start_time but null end_time and errors > 0, flagged as "crashed" and imputed end_time by adding 30 seconds.
Standardized categorical columns (device, geo), replacing invalid or missing values with "unknown".
Standardized bitrate values against allowed list; replaced invalid values with "unknown".
Cleaned errors column: null/blank set to "no_error", invalid entries mapped to "crash".

Computed session duration (watch_span) where possible.
Saved the cleaned and enriched events data to silver table.

5. General Missing Data Handling Approaches:

Replaced missing or invalid categorical values with "unknown" or default values (e.g., "no_error").

Dropped rows with critical missing identifiers.

Imputed missing end_time for crashed sessions with a 30-second window.

Replaced invalid numeric values with 0 or average values to handle outliers.

Trimmed strings to avoid issues from leading/trailing spaces.

```
catalog = "aks_main"
from pyspark.sql.functions import col, trim, lower, when

# Step 1: Load Bronze Table
bronze_users = spark.table(f"{catalog}.bronze.bronze_users_csv")

# Step 2: Clean Data
silver_users = (
    bronze_users
    # Trim whitespaces from all string columns
    .withColumn("user_id", trim(col("user_id")))
    .withColumn("plan_type", trim(col("plan_type")))
    .withColumn("billing_cycle", trim(col("billing_cycle")))
    .withColumn("payment_status", trim(col("payment_status")))
    .withColumn("churn_flag", trim(col("churn_flag")))

    # Drop invalid user_id
    .filter((col("user_id").isNotNull()) &
    (~lower(col("user_id")).startswith("invalid")))

    # Standardize plan_type and replace invalid/missing with 'unknown'
    .withColumn(
        "plan_type",
        when(lower(col("plan_type")).isin("basic", "standard", "premium"),
        col("plan_type"))
        .otherwise("unknown")
    )

    # Standardize billing_cycle
    .withColumn(
        "billing_cycle",
        when(lower(col("billing_cycle")).isin("monthly", "yearly"),
        col("billing_cycle"))
        .otherwise("unknown")
    )
)
```

```

# Standardize payment_status
.withColumn(
    "payment_status",
    when(lower(col("payment_status")).isin("active", "cancelled", "delinquent"), lower(col("payment_status")))
    .otherwise("unknown")
)

# Standardize churn_flag: Active → 0, others → 1, missing/invalid → 0
.withColumn(
    "churn_flag",
    when(lower(col("payment_status")) == "active", 0)
    .otherwise(1)
)

# Step 3: Write Silver Table

silver_users_cleaned = silver_users.drop("ingest_file")

# Overwrite the table and remove column completely
silver_users_cleaned.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.silver.silver_users")

```

```

silver_users_casted = silver_users.select(
    col("user_id").cast("string"),
    col("plan_type").cast("string"),
    col("billing_cycle").cast("string"),
    col("payment_status").cast("string"),
    col("churn_flag").cast("int") # cast churn_flag to integer
)

silver_users_casted.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.silver.silver_users")

```

```

from pyspark.sql.functions import col, trim, when, lower

```

```

# Load bronze content table
bronze_content = spark.table(f"{catalog}.bronze.bronze_content_csv")

# Clean content data
silver_content = (
    bronze_content
    # Trim whitespaces
    .withColumn("content_id", trim(col("content_id")))
    .withColumn("title", trim(col("title")))
    .withColumn("genre", trim(col("genre")))
    .withColumn("duration_min", trim(col("duration_min")))
    .withColumn("region_availability",
trim(col("region_availability")))

    # Drop invalid content_id (null or starts with "invalid")
    .filter((col("content_id").isNotNull()) &
(~lower(col("content_id")).startswith("invalid")))

    # Replace invalid/null/blank title with 'unknown'
    .withColumn(
        "title",
        when(
            col("title").isNull() | (col("title") == "") |
lower(col("title")).startswith("invalid"),
            "unknown"
        ).otherwise(col("title"))
    )

    # Replace invalid/null/blank genre with 'unknown'
    .withColumn(
        "genre",
        when(
            col("genre").isNull() | (col("genre") == "") |
lower(col("genre")).startswith("invalid"),
            "unknown"
        ).otherwise(col("genre"))
    )

    # Replace null/blank/negative/invalid duration with 0, cast to int
    .withColumn(
        "duration_min",

```



```

        when(col("duration_min").rlike("[0-9]+$"),
col("duration_min").cast("int")).otherwise(0)
    )

    # Remove outliers: duration > 300 minutes
    .filter(col("duration_min") <= 300)

    # Replace invalid/null/blank region_availability with 'unknown'
    .withColumn(
        "region_availability",
        when(
            col("region_availability").isNull() |
(col("region_availability") == "") |
lower(col("region_availability")).startswith("invalid"),
            "unknown"
        ).otherwise(col("region_availability"))
    )

    # Drop the ingest_file column if exists
    .drop(*[c for c in ["ingest_file"] if c in bronze_content.columns])
)

# Write cleaned silver content table
silver_content.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.silver.silver_content")

```

```

from pyspark.sql import functions as F
from pyspark.sql.functions import col, trim, when, avg, stddev, lower

catalog = "aks_main"

# Load bronze ads table
bronze_ads = spark.table(f"{catalog}.bronze.bronze_ads_csv")

# -----
# Trim all string columns
# -----
ads_trimmed = bronze_ads.select(
    trim(col("ad_id")).alias("ad_id"),
    trim(col("campaign")).alias("campaign"),

```

```

    trim(col("content_id")).alias("content_id"),
    trim(col("impressions")).alias("impressions"),
    trim(col("clicks")).alias("clicks"),
    trim(col("revenue")).alias("revenue")
)

# -----
# Remove rows with invalid/null/blank values
# -----
ads_cleaned = ads_trimmed.filter(
    (col("ad_id").isNotNull()) & (col("ad_id") != "") &
    (~lower(col("ad_id")).startswith("invalid")) &
    (col("campaign").isNotNull()) & (col("campaign") != "") &
    (~lower(col("campaign")).startswith("invalid"))
).withColumn(
    "content_id", when(col("content_id").isNull() | (col("content_id")
== "")) | lower(col("content_id")).startswith("invalid"),
                    "unknown").otherwise(col("content_id"))
).withColumn(
    "revenue", when(col("revenue").rlike(r"^[0-9]+(\.[0-9]+)?$"),
col("revenue").cast("float")).otherwise(0)
).withColumn(
    "impressions", when(col("impressions").rlike(r"^[0-9]+$") &
(col("impressions").cast("int") >= 0),
                    col("impressions").cast("int")).otherwise(0)
).withColumn(
    "clicks", when(col("clicks").rlike(r"^[0-9]+$") &
(col("clicks").cast("int") >= 0),
                    col("clicks").cast("int")).otherwise(0)
)

# Ensure clicks <= impressions
ads_cleaned = ads_cleaned.withColumn(
    "clicks", when(col("clicks") > col("impressions"),
col("impressions")).otherwise(col("clicks"))
)

# Compute averages and standard deviations for outlier handling
impr_avg, click_avg = ads_cleaned.select(avg(col("impressions")),
avg(col("clicks"))).first()
impr_std, click_std = ads_cleaned.select(stddev(col("impressions")),
stddev(col("clicks"))).first()

```

```

# Replace outliers with mean if they deviate > 3*std
ads_final = ads_cleaned.withColumn(
    "impressions", when(F.abs(col("impressions") - impr_avg) >
3*impr_std, impr_avg).otherwise(col("impressions"))
).withColumn(
    "clicks", when(F.abs(col("clicks") - click_avg) > 3*click_std,
click_avg).otherwise(col("clicks"))
)

# Drop ingest_file if exists
if "ingest_file" in ads_final.columns:
    ads_final = ads_final.drop("ingest_file")

# Write to Silver table
ads_final.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.silver.silver_ads")

# Base paths in ADLS
base_path =
"abfss://akscont1234@aksssstore.dfs.core.windows.net/raw_data"
schema_base_path =
"abfss://akscont1234@aksssstore.dfs.core.windows.net/schemas"
chkpt_base_path =
"abfss://akscont1234@aksssstore.dfs.core.windows.net/chkpts"

from pyspark.sql.functions import col
from pyspark.sql.types import *

# -----
# Events CSV - streaming with trigger once
# -----

# Define schema for Events (all as StringType in Bronze)
events_schema = StructType([
    StructField("event_id", StringType(), True),
    StructField("user_id", StringType(), True),
    StructField("content_id", StringType(), True),
    StructField("start_time", StringType(), True),
    StructField("end_time", StringType(), True),
    StructField("device", StringType(), True),
    StructField("geo", StringType(), True),

```

```

        StructField("bitrate", StringType(), True),
        StructField("errors", StringType(), True)
    ])

bronze_events_csv = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
f"{schema_base_path}/events_csv")
    .schema(events_schema)
    .load(f"{base_path}/events_data")
    .withColumn("ingest_file", col("_metadata.file_path"))
)

query_events_csv = (bronze_events_csv.writeStream
    .format("delta")
    .option("checkpointLocation", f"{chkpt_base_path}/events_csv")
    .outputMode("append")
    .trigger(once=True)
    .toTable(f"{catalog}.bronze.bronze_events_csv")
)

from pyspark.sql import functions as F
from pyspark.sql.functions import col, trim, when, to_timestamp, lower,
lit, coalesce

catalog = "aks_main"
bronze = spark.table(f"{catalog}.bronze.bronze_events_csv")

ALLOWED_BITRATES = [240, 360, 480, 720, 1080, 2160]

# -----
# 1) Trim columns
# -----

df = bronze.select(
    trim(col("event_id")).alias("event_id"),
    trim(col("user_id")).alias("user_id"),
    trim(col("content_id")).alias("content_id"),
    trim(col("start_time")).alias("start_time_str"),
    trim(col("end_time")).alias("end_time_str"),
    trim(col("device")).alias("device"),
    trim(col("geo")).alias("geo"),

```

```

        trim(col("bitrate")).alias("bitrate_str"),
        trim(col("errors")).alias("errors")
    )

# -----
# 2) Drop unusable rows (invalid IDs)
# -----
df = df.filter(
    ~(col("event_id").isNull() |
lower(col("event_id")).startswith("invalid")) &
    ~(col("user_id").isNull() |
lower(col("user_id")).startswith("invalid")) &
    ~(col("content_id").isNull() |
lower(col("content_id")).startswith("invalid"))
)

# -----
# 3) Parse timestamps, drop rows with both missing
# -----
df = df.withColumn(
    "start_time",
    coalesce(
        to_timestamp(col("start_time_str"), "yyyy-MM-dd HH:mm:ss"),
        to_timestamp(col("start_time_str"), "yyyy/MM/dd HH:mm:ss"),
        to_timestamp(col("start_time_str"))
    )
).withColumn(
    "end_time",
    coalesce(
        to_timestamp(col("end_time_str"), "yyyy-MM-dd HH:mm:ss"),
        to_timestamp(col("end_time_str"), "yyyy/MM/dd HH:mm:ss"),
        to_timestamp(col("end_time_str"))
    )
).drop("start_time_str", "end_time_str")

# Drop rows with no valid time at all
df = df.filter(~(col("start_time").isNull() &
col("end_time").isNull()))
df = df.filter(~(col("start_time").isNull() ))

# -----
# 4) Handle crashed sessions (start_time valid, end_time null, errors >
0)

```

```

# -----
df = df.withColumn(
    "session_status",
    when((col("start_time").isNotNull()) & (col("end_time").isNull()) &
(col("errors") > 0),
        lit("crashed"))
    .otherwise(lit("completed"))
)

df = df.withColumn(
    "end_time",
    when((col("session_status") == "crashed"),
        (col("start_time").cast("long") + lit(30)).cast("timestamp"))
# impute 30s
    .otherwise(col("end_time"))
)

# -----
# 5) Standardize categorical fields
# -----
df = df.withColumn(
    "device",
    when(col("device").isNull() | (col("device") == "") |
lower(col("device")).startswith("invalid"),
        "unknown").otherwise(col("device"))
).withColumn(
    "geo",
    when(col("geo").isNull() | (col("geo") == "") |
lower(col("geo")).startswith("invalid"),
        "unknown").otherwise(col("geo"))
)

# -----
# 6) Standardize bitrate
# -----
df = df.withColumn("bitrate_int", col("bitrate_str").cast("int"))
df = df.withColumn(
    "bitrate",
    when(col("bitrate_int").isin(ALLOWED_BITRATES),
col("bitrate_int")).otherwise(lit("unknown"))
).drop("bitrate_str", "bitrate_int")

# -----

```

```

# 7) Standardize errors (string categories)
# -----
VALID_ERRORS = ["network_drop", "buffering", "crash"]

df = df.withColumn(
    "errors",
    when(col("errors").isNull() | (trim(col("errors")) == ""),
lit("no_error")) # null/blank → no_error
    .when(lower(col("errors")).isin([e.lower() for e in VALID_ERRORS]),
lower(col("errors"))) # keep valid
    .when(lower(col("errors")).like("invalid%"), lit("crash")) #
invalidX → crash
    .otherwise(lit("crash")) # everything else → crash
)

# -----
# 8) Optional: compute session duration
# -----
df = df.withColumn(
    "watch_span",
    when((col("start_time").isNotNull()) &
(col("end_time").isNotNull()),
        col("end_time").cast("long") - col("start_time").cast("long"))
)

# -----
# 9) Save to Silver
# -----
df.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.silver.silver_events")

```

```

from pyspark.sql import functions as F

catalog = "aks_main"

# Load Silver Events
events = spark.table(f"{catalog}.silver.silver_events")

```

10) Gold layer code - Operations Dashboard

```

from pyspark.sql import functions as F

gold_concurrent_viewers = (
    events
    .withWatermark("start_time", "5 minutes")
    .groupBy(
        F.window("start_time", "5 minutes"),
        "content_id"
    )
    .agg(
        F.countDistinct("user_id").alias("concurrent_viewers")
    )
    .withColumn("window_start", F.col("window").start)
    .withColumn("window_end", F.col("window").end)
    .drop("window")
)

gold_concurrent_viewers.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_concurrent_viewers")

```

```

gold_stream_quality_fact = (
    events
    .withColumn("day", F.to_date("start_time"))
    .groupBy("day", "content_id", "geo", "device")
    .agg(
        F.avg("bitrate").alias("avg_bitrate"),
        F.sum(F.when(F.col("errors") == "buffering",
1).otherwise(0)).alias("buffering_count"),
        F.sum(F.when(F.col("errors") == "crash",
1).otherwise(0)).alias("crash_count"),
        F.sum(F.when(F.col("errors") == "network_drop",
1).otherwise(0)).alias("network_drop_count")
    )
)

gold_stream_quality_fact.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable(f"{catalog}.gold.gold_stream_quality_fact")

```

```

from pyspark.sql import functions as F

```



```

gold_device_usage = (
    # Start with the silver layer table
    spark.table(f"{catalog}.silver.silver_events")

    # 1. Add the 'day' column from the start_time
    .withColumn("day", F.to_date("start_time"))

    # Add a filter to drop records where 'day' is null.
    .filter(F.col("day").isNotNull())

    # 2. Clean the 'device' column
    .withColumn("device", F.lower(F.col("device")))
    .withColumn(
        "device",
        F.when(
            F.lower(F.col("device")).isin("invalid", "n/a", "unknown",
"null"),
            "unknown"
        ).otherwise(F.col("device"))
    )

    # 3. Calculate 'watch_time_sec' only for valid sessions
    .withColumn(
        "watch_time_sec",
        F.when(
            F.col("end_time").isNotNull() & (F.col("end_time") >=
F.col("start_time")),
            F.col("end_time").cast("long") -
F.col("start_time").cast("long")
        )
        .otherwise(F.lit(0))
    )

    # 4. Group by day and device to perform aggregations
    .groupBy("day", "device")
    .agg(
        F.countDistinct("user_id").alias("active_users"),
        F.sum("watch_time_sec").alias("total_watch_time_sec")
    )
)

gold_device_usage.write.format("delta") \

```

```
.mode("overwrite") \
.saveAsTable(f"{catalog}.gold.gold_device_usage")
```

```
gold_geo_usage = (
    events
    .withColumn("day", F.to_date("start_time"))
    .withColumn("watch_time_sec", F.col("end_time").cast("long") -
F.col("start_time").cast("long"))
    .groupBy("day", "geo")
    .agg(
        F.countDistinct("user_id").alias("active_users"),
        F.sum("watch_time_sec").alias("total_watch_time_sec"),
        F.count("*").alias("total_sessions")
    )
)

gold_geo_usage.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable(f"{catalog}.gold.gold_geo_usage")
```

```
from pyspark.sql import functions as F

# The correct approach is to start with a clean, standardized Silver
table.
silver_events = spark.table(f"{catalog}.silver.silver_events")

gold_error_fact = (
    silver_events
    # 1. Derive the 'day' column from the clean 'start_time' and drop
nulls.
    .withColumn("day", F.to_date("start_time"))
    .filter(F.col("day").isNotNull())

    # 2. Filter out records that are not errors before aggregation.
    # We use a filter to exclude any value that indicates 'no error'
    .filter(~F.lower(F.col("errors")).isin("no_error", "none"))

    # 3. Group by the now clean dimensions.
    .groupBy("day", "errors", "geo", "device")
    .agg(
        F.count("*").alias("error_occurrences"),
        F.countDistinct("user_id").alias("unique_users_affected")
    )
)
```

```
)  
)  
  
gold_error_fact.write.format("delta") \  
    .mode("overwrite") \  
    .option("overwriteSchema", "true") \  
    .saveAsTable(f"{catalog}.gold.gold_error_fact")
```

1. Concurrent Viewers per Content (gold_concurrent_viewers)

Groups events into 5-minute time windows by content.

Counts distinct users watching each content during each window.

Insight: Measures the number of concurrent viewers per content over time, useful for understanding peak demand and content popularity in short intervals.

2. Stream Quality Metrics (gold_stream_quality_fact)

Aggregates data by day, content, geographic region (geo), and device type.

Calculates average bitrate (video quality indicator).

Counts occurrences of different streaming errors: buffering, crashes, network drops.

Insight: Provides a daily quality-of-experience summary per content/region/device, helping identify performance issues or user experience problems.

3. Device Usage Summary (gold_device_usage)

Groups by day and device type after cleaning device names.

Counts distinct active users per device daily.

Calculates total watch time (in seconds) per device per day.

Insight: Shows user engagement and viewing patterns across different device types, which helps prioritize device-specific optimizations or marketing.

4. Geographic Usage Summary (gold_geo_usage)

Groups by day and geographic region.

Counts distinct active users and total viewing sessions.

Sums total watch time in seconds for each region daily.

Insight: Provides a regional overview of user engagement and content consumption, useful for geo-targeted strategies or infrastructure planning.

5. Streaming Error Facts (gold_error_fact)

Focuses only on records with actual streaming errors (excluding 'no_error').
Groups by day, error type, region, and device.
Counts total error occurrences and unique users affected by each error.
Insight: Enables monitoring of streaming issues by error type, region, and device, helping identify and prioritize fixes for the most impactful problems.

6. General Purpose & Insights:

These transformations aggregate and clean event data from the Silver layer into actionable metrics.

Enable business users to track user behavior, content popularity, device usage, streaming quality, and error trends over time.

Help in making data-driven decisions to improve user experience, optimize content delivery, and troubleshoot streaming issues effectively.

11) Gold layer code - Product Dashboard

```
from pyspark.sql import functions as F

# -----
# Load Silver Tables
# -----

users = spark.table(f"{catalog}.silver.silver_users")
content = spark.table(f"{catalog}.silver.silver_content")
events = spark.table(f"{catalog}.silver.silver_events")

# Add useful derived columns
events = (
    events
    .withColumn("day", F.to_date("start_time"))
    .withColumn("watch_minutes", F.col("watch_span")/60.0) # convert
seconds → minutes
)

# =====
# 1. gold_content_dim → Content dimension
# =====

gold_content_dim = (
    content
    .select(
        "content_id",
        "title",
```

```

        "genre",
        "duration_min",
        "region_availability"
    )
    .dropDuplicates(["content_id"])
    .withColumnRenamed("duration_min", "duration")
    .withColumnRenamed("region_availability", "region")
)

gold_content_dim.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_content_dim")

# =====
# 2. gold_content_engagement_fact → Engagement metrics
# =====

gold_content_engagement_fact = (
    events
    .join(content, "content_id", "left")
    .groupBy("content_id", "region_availability", "day")
    .agg(
        F.sum("watch_minutes").alias("total_watch_hours"),
        F.avg("watch_minutes").alias("avg_watch_minutes"),
        F.countDistinct("user_id").alias("unique_viewers")
    )
    .withColumn("total_watch_hours", F.col("total_watch_hours")/60.0)
# minutes → hours
    .withColumnRenamed("region_availability", "region")
)

gold_content_engagement_fact.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_content_engagement_fact")

# =====
# 3. gold_genre_trends → Genre-level watch trends
# =====

```

```

gold_genre_trends = (
    events
    .join(content, "content_id", "left")
    .groupBy("genre", "day")
    .agg(
        F.sum("watch_minutes").alias("total_watch_hours"),
        F.countDistinct("user_id").alias("unique_viewers")
    )
    .withColumn("total_watch_hours", F.col("total_watch_hours")/60.0)
)

```

```

gold_genre_trends.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_genre_trends")

```

```

# =====
# 4. gold_user_retention_fact → D1/D7/D30 retention
# =====

```

```

first_watch = (
    events.groupBy("user_id")
    .agg(F.min("day").alias("signup_day"))
)

```

```

retention = (
    events.join(first_watch, "user_id")
    .withColumn("days_since_signup", F.datediff("day", "signup_day"))
)

```

```

gold_user_retention_fact = (
    retention
    .join(content, "content_id", "left")
    .groupBy("genre", "days_since_signup")
    .agg(F.countDistinct("user_id").alias("retained_users"))
    .filter(F.col("days_since_signup").isin([1, 7, 30]))
)

```

```

gold_user_retention_fact.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_user_retention_fact")

```

```

# =====
# 5. gold_cross_device_behavior → User device usage
# =====

gold_cross_device_behavior = (
    events
    .groupBy("user_id", "day")
    .agg(F.collect_set("device").alias("devices_used"))
    .withColumn("num_devices", F.size("devices_used"))
)

gold_cross_device_behavior.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_cross_device_behavior")

```

```

from pyspark.sql import functions as F
from pyspark.sql.functions import col, countDistinct, when, lit

catalog = "aks_main"

# Load Silver tables
users    = spark.table(f"{catalog}.silver.silver_users")
events   = spark.table(f"{catalog}.silver.silver_events")
ads      = spark.table(f"{catalog}.silver.silver_ads")
content  = spark.table(f"{catalog}.silver.silver_content")

# =====
# 1. gold_user_dim
# =====
# Take latest region per user (from events.geo)
user_region = (events
    .groupBy("user_id")
    .agg(F.first("geo", ignorenulls=True).alias("region"))
)

gold_user_dim = (users
    .join(user_region, "user_id", "left")
    .select("user_id", "plan_type", "billing_cycle", "churn_flag",
"region")

```

```

)

gold_user_dim.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_user_dim")

# =====
# 2. gold_subscription_fact
# =====

gold_subscription_fact = (users
    .agg(
        F.count(when(col("payment_status") == "active",
1)).alias("active_subs"),
        F.count(when(col("churn_flag") == 1,
1)).alias("churned_users"),
        F.count(when(col("plan_type") == "premium",
1)).alias("premium_users"),
        F.count(when(col("plan_type") == "basic",
1)).alias("basic_users"),
        F.count(when(col("plan_type") == "standard",
1)).alias("standard_users")
    )
)

gold_subscription_fact.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_subscription_fact")

# =====
# 3. gold_churn_features (ML-ready)
# =====

watch_features = (events
    .groupBy("user_id")
    .agg(
        F.avg("watch_span").alias("avg_watch_time"),
        F.sum("watch_span").alias("total_watch_time"),
        F.count("*").alias("session_count"),
        F.count(when(col("errors") != "no_error",
1)).alias("error_count"),
        countDistinct("device").alias("distinct_devices")
    )
)

```



```

)

genre_features = (events
    .join(content, "content_id", "left")
    .groupBy("user_id")
    .agg(countDistinct("genre").alias("distinct_genres_watched"))
)

gold_churn_features = (users
    .join(watch_features, "user_id", "left")
    .join(genre_features, "user_id", "left")
    .fillna(0,
["avg_watch_time", "total_watch_time", "session_count", "error_count", "distinct_devices", "distinct_genres_watched"])
    .select("user_id", "plan_type", "billing_cycle", "payment_status",
            "avg_watch_time", "total_watch_time", "session_count",
"error_count",
            "distinct_devices", "distinct_genres_watched",
"churn_flag")
)

gold_churn_features.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_churn_features")

# =====
# 4. gold_ad_campaign_fact
# =====

gold_ad_campaign_fact = (ads
    .groupBy("campaign", "content_id")
    .agg(
        F.sum("impressions").alias("total_impressions"),
        F.sum("clicks").alias("total_clicks"),
        (F.sum("clicks") / F.sum("impressions")).alias("CTR"),
        F.sum("revenue").alias("total_revenue")
    )
)

gold_ad_campaign_fact.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_ad_campaign_fact")

```

```

# =====
# 5. gold_ad_attribution_fact
# =====

gold_ad_attribution_fact = (ads
    .join(events, "content_id", "inner")
    .groupBy("ad_id", "campaign", "content_id", "user_id")
    .agg(
        F.sum("impressions").alias("impressions"),
        F.sum("clicks").alias("clicks"),
        F.sum("revenue").alias("revenue"),
        F.sum("watch_span").alias("watch_time_after_ad")
    )
)

gold_ad_attribution_fact.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_ad_attribution_fact")

# =====
# 6. gold_marketing_funnel
# =====

gold_marketing_funnel = (users
    .agg(
        F.count("*").alias("total_users"),
        F.count(when(col("payment_status") == "active",
1)).alias("active_users"),
        F.count(when((col("payment_status") == "active") &
(col("plan_type") != "unknown"), 1)).alias("paying_users"),
        F.count(when(col("churn_flag") == 1, 1)).alias("churned_users")
    )
    .withColumn("trial_users", lit(0)) # Placeholder, if you add trial
flag later
)

gold_marketing_funnel.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_marketing_funnel")

```

```

from pyspark.sql import functions as F
from pyspark.sql.functions import col, lit

catalog = "aks_maiin"

# Load Silver tables
users = spark.table(f"{catalog}.silver.silver_users")
ads = spark.table(f"{catalog}.silver.silver_ads")
events = spark.table(f"{catalog}.silver.silver_events")

# =====
# 1. gold_revenue_fact
# =====

# --- Subscription revenue (assumption: ARPU mapping by plan) ---
plan_revenue_map = {
    "basic": 5.0,
    "standard": 10.0,
    "premium": 15.0,
    "unknown": 0.0
}

plan_revenue_expr = F.create_map([lit(x) for x in
sum(plan_revenue_map.items(), ())])

subscription_revenue = (users
    .withColumn("subscription_revenue",
plan_revenue_expr[col("plan_type")])
    .withColumn("date", F.current_date()) # Replace with billing date
if available
    .groupBy("date", "plan_type")

    .agg(F.sum("subscription_revenue").alias("total_subscription_revenue"))
)

# --- Ad revenue ---
ad_revenue = (ads
    .withColumn("date", F.current_date()) # Replace with ad_date if
available
    .groupBy("date", "content_id")
    .agg(F.sum("revenue").alias("total_ad_revenue"))
)

# Combine both

```

```

gold_revenue_fact = (subscription_revenue
    .join(ad_revenue, "date", "outer")
    .fillna(0, ["total_subscription_revenue", "total_ad_revenue"])
    .withColumn("total_revenue", col("total_subscription_revenue") +
col("total_ad_revenue"))
)

gold_revenue_fact.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_revenue_fact")

# =====
# 2. gold_user_finance_fact
# =====
# -----
# 2. gold_user_finance_fact (fixed)
# -----

# Subscription revenue per user
user_subs = (users
    .withColumn("subscription_revenue",
plan_revenue_expr[col("plan_type")])
    .select("user_id", "billing_cycle", "subscription_revenue")
)

# Map users → content from events
user_content_map = (events
    .select("user_id", "content_id")
    .distinct()
)

# Join to ads via content_id
user_ads = (user_content_map
    .join(ads.groupBy("content_id")
        .agg(F.sum("revenue").alias("ad_revenue")),
        "content_id", "left")
    .groupBy("user_id")
    .agg(F.sum("ad_revenue").alias("ad_revenue"))
)

# Combine subscription + ad revenue
gold_user_finance_fact = (user_subs

```

```

.join(user_ads, "user_id", "left")
.fillna(0, ["ad_revenue"])
.withColumn("total_revenue", col("subscription_revenue") +
col("ad_revenue"))
.groupBy("user_id", "billing_cycle")
.agg(
    F.first("subscription_revenue").alias("subscription_revenue"),
    F.first("ad_revenue").alias("ad_revenue"),
    F.sum("total_revenue").alias("ltv"), # lifetime value
    (F.sum("total_revenue")/F.count("user_id")).alias("arpu")
)
)

gold_user_finance_fact.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_user_finance_fact")

# =====
# 3. gold_campaign_roi
# =====
# Assume campaign_spend not available → placeholder
gold_campaign_roi = (ads
    .groupBy("campaign")
    .agg(
        F.sum("revenue").alias("campaign_revenue"),
        F.lit(1000.0).alias("campaign_spend") # placeholder
    )
    .withColumn("roi", col("campaign_revenue") / col("campaign_spend"))
)

gold_campaign_roi.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(f"{catalog}.gold.gold_campaign_roi")

```

1. gold_content_dim (Content Dimension)

Contains unique content metadata: content_id, title, genre, duration, region.

Insight: Forms the base dimension table to enrich engagement, retention, and ad data.

2. gold_content_engagement_fact

Aggregates user engagement per content/day/region.

Metrics: total_watch_hours, avg_watch_minutes, unique_viewers.

Insight: Shows how much users are engaging with content and where it's most popular.

3. gold_genre_trends

Groups viewing trends by genre and day.

Metrics: total_watch_hours, unique_viewers.

Insight: Reveals genre popularity over time — helps in content strategy.

4. gold_user_retention_fact

- Tracks **D1, D7, D30 retention** by genre.
- Calculates how many users return to watch content after signup.
-  **Insight:** Measures user stickiness and long-term engagement by content type.

5. gold_cross_device_behavior

- Analyzes how many different devices users watch on per day.
-  **Insight:** Highlights cross-device behavior — useful for understanding multi-platform usage.

6. gold_user_dim (User Dimension)

- Combines user profile with inferred region from usage.
-  **Insight:** Standard user dimension for joining across facts — helpful for segmentation.

7. gold_subscription_fact

Summary of user subscriptions: Active vs churned & Basic, Standard, Premium plan counts

Insight: Tracks overall subscription health and product tier adoption.

8. gold_churn_features

Builds **ML-ready features** for churn prediction:

Watch time, sessions, errors, device diversity, genre diversity

Insight: Enables churn modeling and user retention prediction.

9. gold_ad_campaign_fact

Ad performance summary per campaign and content.

Metrics: impressions, clicks, CTR (Click Through Rate), revenue.

Insight: Measures effectiveness of ad campaigns by engagement and monetization.

10. gold_ad_attribution_fact

Maps ads to content/user engagement:

How much a user watched *after* seeing an ad.

Insight: Tracks ad impact on user engagement and monetization per user.

11. gold_marketing_funnel

High-level counts:

Total users, active, paying, churned (trial placeholder)

Insight: Tracks conversion funnel from signup to paid user to churn.

12. gold_revenue_fact

Combines: Estimated **subscription revenue** (via ARPU by plan), **Ad revenue** from impressions/clicks & Final total_revenue per day

Insight: Daily revenue tracking — split by ads and subscriptions.

13. gold_user_finance_fact

Revenue metrics **per user**: Subscription + ad revenue, ARPU (Average Revenue Per User), LTV (Lifetime Value)

Insight: User-level monetization tracking — vital for LTV prediction and ROI.

14. gold_campaign_roi

Calculates **Return on Investment** per campaign.

Assumes a fixed spend placeholder for now.

Insight: Measures profitability of marketing campaigns ($ROI = \text{Revenue} / \text{Spend}$).

Summary of Insights

Engagement: What content, genre, or region drives most watch time and attention?

Retention: Are users coming back? How does this vary by genre?

Device Behavior: How many users switch devices daily?

Monetization: How much revenue is being generated via subscriptions and ads?

User Segmentation: How do different user types (plan, region) behave?

Churn Modeling: What behavioral factors signal churn risk?

Ad Effectiveness: Which campaigns are driving value?

Revenue Attribution: Which users or content generate the most money?

12) Masking

```
%sql
CREATE OR REPLACE VIEW aks_main.silver.masked_silver_users AS
SELECT
    CASE
        WHEN current_user() = 'admin' THEN user_id
        WHEN current_user() IN ('data_scientist','data_engineer') THEN
sha2(user_id, 256)
        ELSE '***MASKED***'
    END AS user_id,
    plan_type,
    billing_cycle,
    CASE
        WHEN current_user() IN ('admin','data_scientist') THEN
payment_status
        ELSE '***MASKED***'
    END AS payment_status,
    churn_flag
FROM aks_main.silver.silver_users;
```

```
%sql
CREATE OR REPLACE VIEW aks_main.silver.masked_silver_events AS
SELECT
    CASE
        WHEN current_user() = 'admin' THEN event_id
        WHEN current_user() IN ('data_scientist','data_engineer') THEN
sha2(event_id, 256)
        ELSE '***MASKED***'
    END AS event_id,
    CASE
        WHEN current_user() = 'admin' THEN user_id
        WHEN current_user() IN ('data_scientist','data_engineer') THEN
sha2(user_id, 256)
        ELSE '***MASKED***'
    END AS user_id,
    content_id,
    start_time,
    end_time,
    device,
```



```

    geo,
    bitrate,
    errors,
    session_status,
    watch_span
FROM aks_main.silver.silver_events;

```

```

%sql
CREATE OR REPLACE VIEW aks_main.gold.masked_gold_user_dim AS
SELECT
    CASE
        WHEN current_user() = 'admin' THEN user_id
        WHEN current_user() IN ('data_scientist','data_engineer') THEN
sha2(user_id,256)
        ELSE '***MASKED***'
    END AS user_id,
    plan_type,
    billing_cycle,
    churn_flag,
    region
FROM aks_main.gold.gold_user_dim;

```

1. Role-Based Masking

Sensitive fields (user_id, event_id, payment_status) are masked differently based on current_user():

admin: Full access to all fields. data_scientist / data_engineer: IDs are **hashed** using sha2() for anonymization.

Other users: Values are replaced with '***MASKED***'.

2. Three Views Are Created:

a) masked_silver_users

Masks user_id and payment_status based on role.

Keeps fields like plan_type, billing_cycle, churn_flag unmasked for all users.

b) masked_silver_events

Masks both event_id and user_id.

Allows full access to engagement fields (e.g., bitrate, watch_span, geo) for all users.

c) masked_gold_user_dim

Masks user_id only. Keeps useful user metadata (plan_type, billing_cycle, churn_flag, region) unmasked.

13) ML code for model experimentation, versioning, and serving

```
from pyspark.sql import functions as F

# -----
# ① Load masked Gold/Silver tables
# -----
users = spark.table("aks_main.gold.masked_gold_user_dim")
events = spark.table("aks_main.silver.masked_silver_events")
ads = spark.table("aks_main.gold.gold_ad_attribution_fact")

# -----
# ② Engagement Features
# -----
engagement = events.groupBy("user_id").agg(
    F.count("event_id").alias("session_count"),
    F.sum("watch_span").alias("total_watch_time_sec"),
    (F.sum("watch_span") /
F.count("event_id")).alias("avg_watch_time_sec"),
    F.countDistinct("device").alias("distinct_devices"),
    F.countDistinct("content_id").alias("distinct_content_watched"),
    F.countDistinct("errors").alias("distinct_error_types")
)

# -----
# ③ Error / Quality Features
# -----
error_features =
events.groupBy("user_id").pivot("errors").count().na.fill(0)
# Example columns: crash, buffering, network_drop, no_error

# Avg bitrate per user
bitrate_avg =
events.groupBy("user_id").agg(F.avg("bitrate").alias("avg_bitrate"))

# -----
# ④ Ad Engagement Features
# -----
ad_features = ads.groupBy("user_id").agg(
    F.sum("impressions").alias("ad_impressions"),
    F.sum("clicks").alias("ad_clicks"),
    F.sum("revenue").alias("ad_revenue"),
    (F.sum("clicks") / F.sum("impressions")).alias("ad_ctr")
)
```

```

)

# -----
# ⑤ Combine All Features
# -----
ml_features = users.join(engagement, "user_id", "left") \
                    .join(error_features, "user_id", "left") \
                    .join(bitrate_avg, "user_id", "left") \
                    .join(ad_features, "user_id", "left") \
                    .fillna(0) # Fill nulls for users with no
events/ads

# -----
# ⑥ Add Target Variable
# -----
ml_features = ml_features.withColumnRenamed("churn_flag", "label") #
0/1

# -----
# ⑦ Null-Safe Manual Categorical Encoding
# -----
categorical_cols = ["plan_type", "billing_cycle", "region"]

for col_name in categorical_cols:
    # Replace nulls with 'unknown'
    ml_features = ml_features.withColumn(col_name,
F.coalesce(F.col(col_name), F.lit("unknown")))

    # Get distinct categories
    categories = [row[0] for row in
ml_features.select(col_name).distinct().collect()]

    # Create integer mapping
    mapping_expr = F.create_map([F.lit(x) for pair in zip(categories,
range(len(categories))) for x in pair])

    ml_features = ml_features.withColumn(f"{col_name}_idx",
mapping_expr[F.col(col_name)])

    ml_features = ml_features.withColumn(f"{col_name}_{cat}",
F.when(F.col(col_name) == cat, 1).otherwise(0))

# -----

```

```
# ⑧ Save ML-Ready Delta Table
# -----
ml_features.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("aks_main.gold.gold_churn_features_ml")

print("✅ ML-ready feature table created:
hetul_catalog_ott_02.gold.gold_churn_features_ml")
```

14) Downstream Power BI access for marketing/finance/product teams

1. Loads Masked Gold & Silver Tables

Uses **masked views** (masked_gold_user_dim, masked_silver_events) to ensure sensitive fields like user_id are hashed or masked depending on user role. Also loads the gold-level ad attribution table.

2. User Engagement Features

Calculates metrics per user: session_count, total_watch_time_sec, avg_watch_time_sec, Number of distinct devices, content, and error types encountered.

3. Error & Quality Features

Pivots error types (e.g., crash, buffering) into separate columns with counts. Computes average bitrate per user (a measure of stream quality).

4. Ad Interaction Features

Aggregates ad-related user metrics: ad_impressions, ad_clicks, ad_revenue, CTR (Click-Through Rate)

5. Combine All Features

Joins all feature sets together using user_id as the key. Fills any missing values (e.g., users with no ads/events) with zero.

6. Adds Target Variable

Renames churn_flag → label to make it ML-compatible (used as the target in classification models).

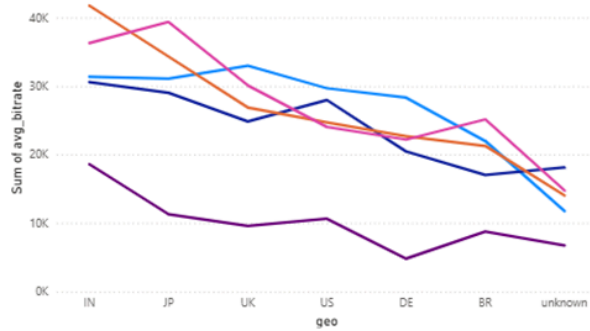
7. Encodes Categorical Variables

Null-safe encoding for plan_type, billing_cycle, and region.

Applies both: **Integer encoding** & **Manual one-hot encoding**

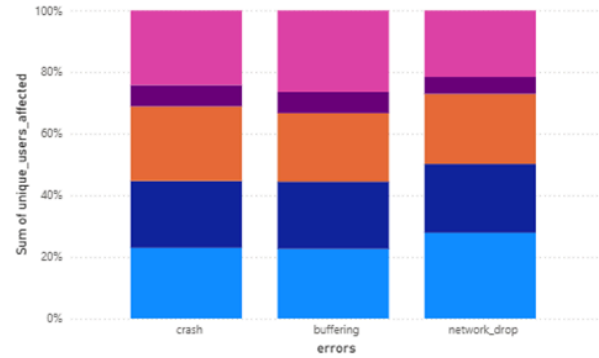
Sum of avg_bitrate by geo and device

device ● Mobile ● SmartTV ● Tablet ● unknown ● Web

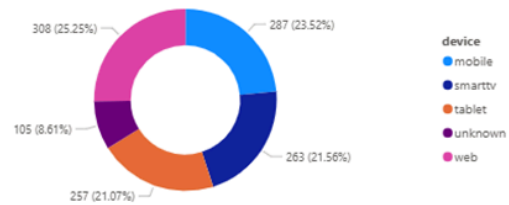


Sum of unique_users_affected by errors and device

device ● Mobile ● SmartTV ● Tablet ● unknown ● Web

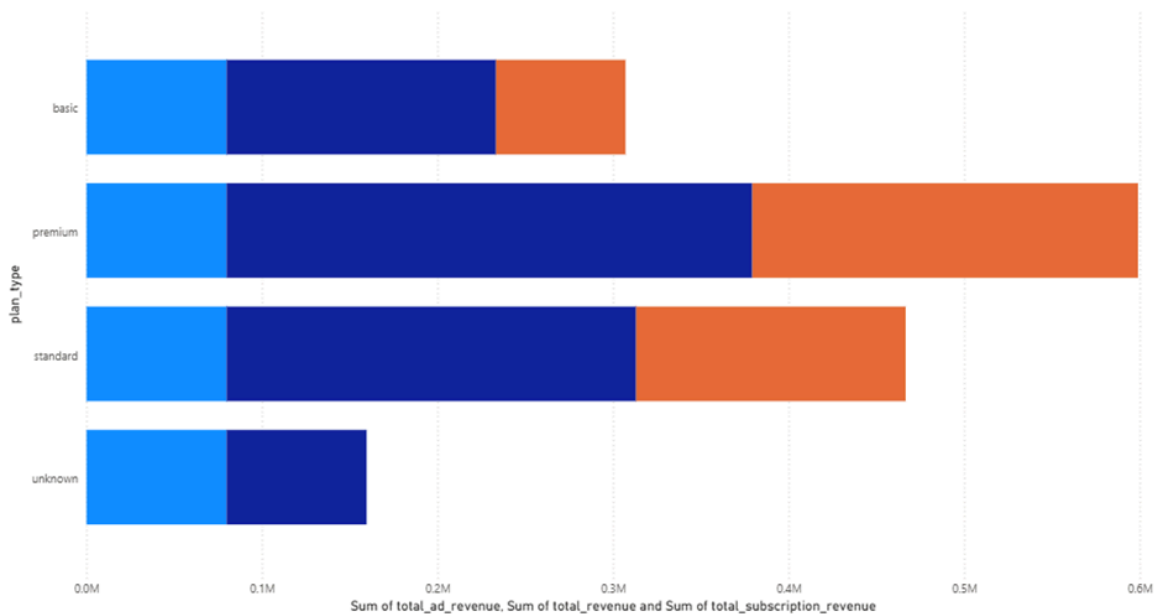


Sum of active_users by device



Sum of total_ad_revenue, Sum of total_revenue and Sum of total_subscription_revenue by plan_type

● Sum of total_ad_revenue ● Sum of total_revenue ● Sum of total_subscription_revenue



15) Conclusion

1. A live operational dashboard with concurrent viewers, stream health, and engagement KPIs.
2. Curated datasets in a governed SQL layer for self-service analytics across marketing, finance, and product.
3. Production-ready churn prediction service with evaluation metrics and drift monitoring.
4. Governance documentation: data catalog, access model, lineage, and retention compliance.
5. Operational runbooks for ingestion, pipeline failures, and model retraining.